Assignment 8 - Big Data Analytics with Docker container

Phạm Trần Gia Hưng – 2331200153

Objective: In this assignment, you will build a complete local Big Data Analytics platform using Docker instead of AWS EMR. This assignment works on: Windows, macOS Intel, macOS Apple Silicon (M1/M2/M3). You will perform the following task listed below:

- Install Docker
- Build Hadoop + Spark Cluster
- Run Spark Analytics Jobs
- Perform SQL Analytics
- Use Kafka Streaming
- Build Dashboards using Superset
- Use Jupyter Notebook for Analytics
- Store Data in PostgreSQL

AWS Service Vs Docker

AWS Service	Docker/Open Source Replacement
S3	Local mounted volume / MinIO
AWS Glue	PySpark ETL jobs
Athena	Spark SQL / Hive
Kinesis	Apache Kafka
EMR	Spark + Hadoop cluster
QuickSight	Apache Superset

Glue Data Catalog Hive Metastore

TECHNOLOGIES USED

Technology	Purpose
Docker	Containerization

Hadoop HDFS	Distributed Storage
Apache Spark	Big Data Processing

PostgreSQL	Metadata & Warehouse
Kafka	Real-Time Streaming
Jupyter Notebook	Analytics Development
Apache Superset	Dashboard Visualization

Assignment Architecture

CSV Dataset
↓
Spark ETL Processing
↓
Cleaned Data
↓
Spark SQL Analytics
↓
PostgreSQL Warehouse
↓
Superset Dashboards

Deliverables: Each student must submit the following:

Presentation Slides (30 points)
○ Title slide with student name(s), course, and assignment title.
○ Give a clear explanation of each completed task.
○ Screenshots or videos of key steps/results.
○ Lessons learned and challenges encountered.

Presentation Guidelines
● Use professional formatting (legible fonts, consistent colors, organized layout). ● Ensure clarity: avoid overcrowding slides with text; use bullet points and visuals. ● Provide screenshots or videos as evidence of work completed. ● Keep presentations within the time limit (8–11 minutes).
● Be prepared to answer questions about the process and challenges.

Professionalism & Delivery (10 points)
● Confident and clear presentation delivery

- Ability to answer questions from peers/instructor
- Team collaboration and equal participation

Submission Instructions

- Submit slides (PowerPoint or PDF) to Moodle by the deadline.

Task completion (70 points)

SECTION A — WINDOWS USER SETUP GUIDE

TASK 1 — Install Docker Desktop on Windows

- Step 1.1 — Download Docker Desktop

Open: https://www.docker.com/products/docker-desktop/

Download: Docker Desktop for Windows

- Step 1.2 — Install Docker Desktop

During installation:

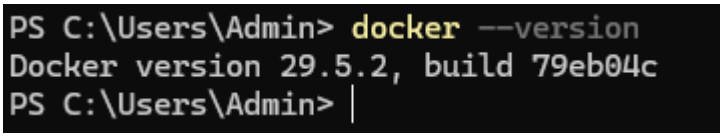
- Enable WSL2 integration
- Enable Hyper-V if asked ● Restart the system after installation.
- Step 1.3 — Verify Docker Installation

Open PowerShell:

- Run: docker --version ● Expected output:

Docker version xx.xx.x

Run: docker compose version



TASK 2 — Create Project Directory

Create project folder:
mkdir bigdata-docker
cd bigdata-docker

Create folders:
mkdir data
mkdir
data\source
mkdir data\target
mkdir notebooks
mkdir spark
mkdir spark\jobs
mkdir jars

```
PS C:\Users\Admin> mkdir bigdata-docker


    Directory: C:\Users\Admin


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----          6/4/2026   6:48 PM                bigdata-docker


PS C:\Users\Admin> cd bigdata-docker
PS C:\Users\Admin\bigdata-docker> mkdir data


    Directory: C:\Users\Admin\bigdata-docker


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----          6/4/2026   6:48 PM                data
```

```
PS C:\Users\Admin\bigdata-docker> mkdir data\target


    Directory: C:\Users\Admin\bigdata-docker\data


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----          6/4/2026   6:49 PM                target

PS C:\Users\Admin\bigdata-docker> mkdir notebooks


    Directory: C:\Users\Admin\bigdata-docker


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----          6/4/2026   6:49 PM                notebooks
```

```
PS C:\Users\Admin\bigdata-docker> mkdir spark

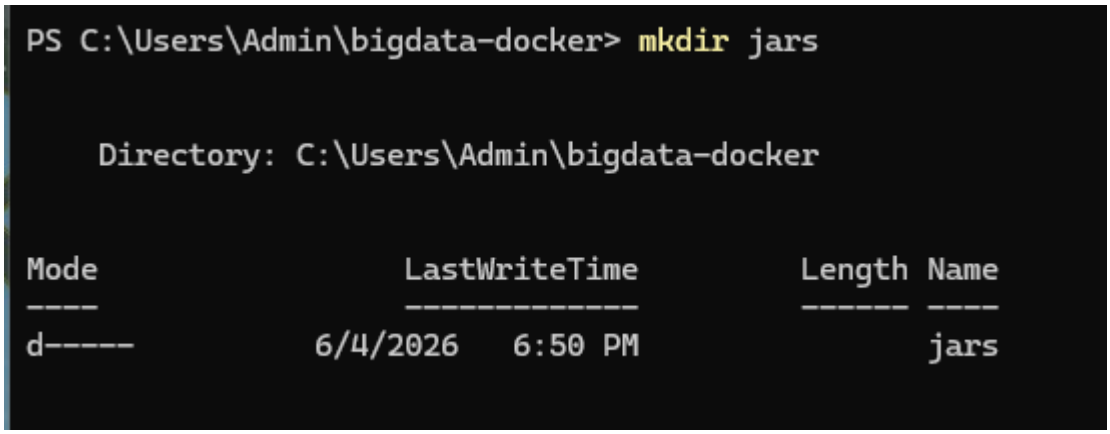

    Directory: C:\Users\Admin\bigdata-docker


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----          6/4/2026   6:49 PM                spark

PS C:\Users\Admin\bigdata-docker> mkdir spark\jobs


    Directory: C:\Users\Admin\bigdata-docker\spark


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----          6/4/2026   6:49 PM                jobs
```



TASK 3 — Create Docker Compose File

Create file: docker-compose.yml

Paste the following code:

```
version: '3.9'

services:

  namenode:
    image: bde2020/hadoop-namenode:2.0.0-hadoop3.2.1-
    java8 container_name: namenode environment:
      CLUSTER_NAME: test
    ports:
      - "9870:9870"
      - "9000:9000"  volumes:
      - namenode:/hadoop/dfs/name

  datanode:
    image: bde2020/hadoop-datanode:2.0.0-hadoop3.2.1-
    java8 container_name: datanode  environment:
      SERVICE_PRECONDITION: namenode:9870
    ports:
      - "9864:9864"
    volumes:
      - datanode:/hadoop/dfs/data  depends_on:
      - namenode

  spark-master:
    image: apache/spark:3.5.1
    platform: linux/amd64
    container_name: spark-master  command:
    >
```

```
    /opt/spark/bin/spark-class
    org.apache.spark.deploy.master.Master
  ports:
    - "8080:8080"
    - "7077:7077"  volumes:
    - ./data:/opt/spark-data
    - ./spark/jobs:/opt/spark/jobs
    - ./jars/postgresql.jar:/opt/spark/jars/postgresql.jar
  environment:
    SPARK_EXTRA_CLASSPATH: /opt/spark/jars/postgresql.jar

spark-worker:
  image: apache/spark:3.5.1
  platform: linux/amd64
  container_name: spark-worker  command:
  >
    /opt/spark/bin/spark-class
    org.apache.spark.deploy.worker.Worke
    r
    spark://spark-
    master:7077
  depends_on:
    - spark-master  volumes:
    - ./data:/opt/spark-data
    - ./spark/jobs:/opt/spark/jobs
    - ./jars/postgresql.jar:/opt/spark/jars/postgresql.jar
  environment:
    SPARK_EXTRA_CLASSPATH: /opt/spark/jars/postgresql.jar

postgres:
  image: postgres:14
  container_name: postgres
  environment:
    POSTGRES_USER: hive
    POSTGRES_PASSWORD: hive
    POSTGRES_DB: metastore
  ports:
    - "5433:5432"

jupyter:
  image: jupyter/pyspark-notebook
  container_name: jupyter
  ports:
    - "8888:8888"  volumes:
    - ./notebooks:/home/jovyan/work  environment:
```

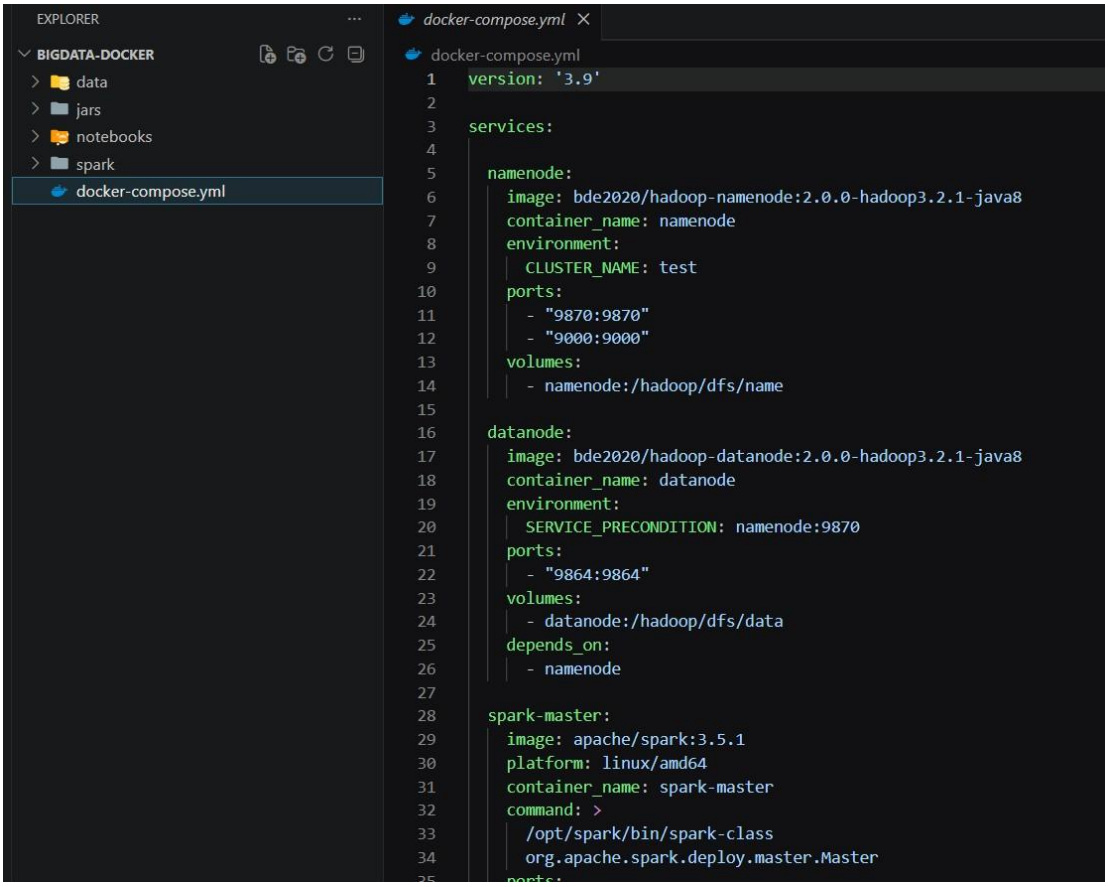
```
    JUPYTER_TOKEN: mypassword

  superset:
    image: apache/superset:latest
    platform: linux/amd64
    container_name: superset
    ports:
      - "8088:8088"  environment:
    SUPERSET_SECRET_KEY: supersecretkey123
    SUPERSET_LOAD_EXAMPLES: "yes"  command:
    >
      /bin/sh -c "  pip install
      psycopg2-binary &&
      superset db upgrade &&
      superset fab create-admin
        --username admin
        --firstname Admin
        --lastname User
        --email admin@example.com
        --password admin123 || true
      &&  superset init &&  superset
      run -h 0.0.0.0 -p 8088
      "

  zookeeper:
    image: confluentinc/cp-zookeeper:7.5.0
    container_name: zookeeper  environment:
      ZOOKEEPER_CLIENT_PORT: 2181
    ports:
      -    "2181:2181"

  kafka:
    image: confluentinc/cp-kafka:7.5.0
    container_name: kafka  depends_on:
      -    zookeeper  ports:
      -    "9092:9092"
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1

volumes:
  namenode:
  datanode:
```



TASK 4 — Download PostgreSQL JDBC Driver

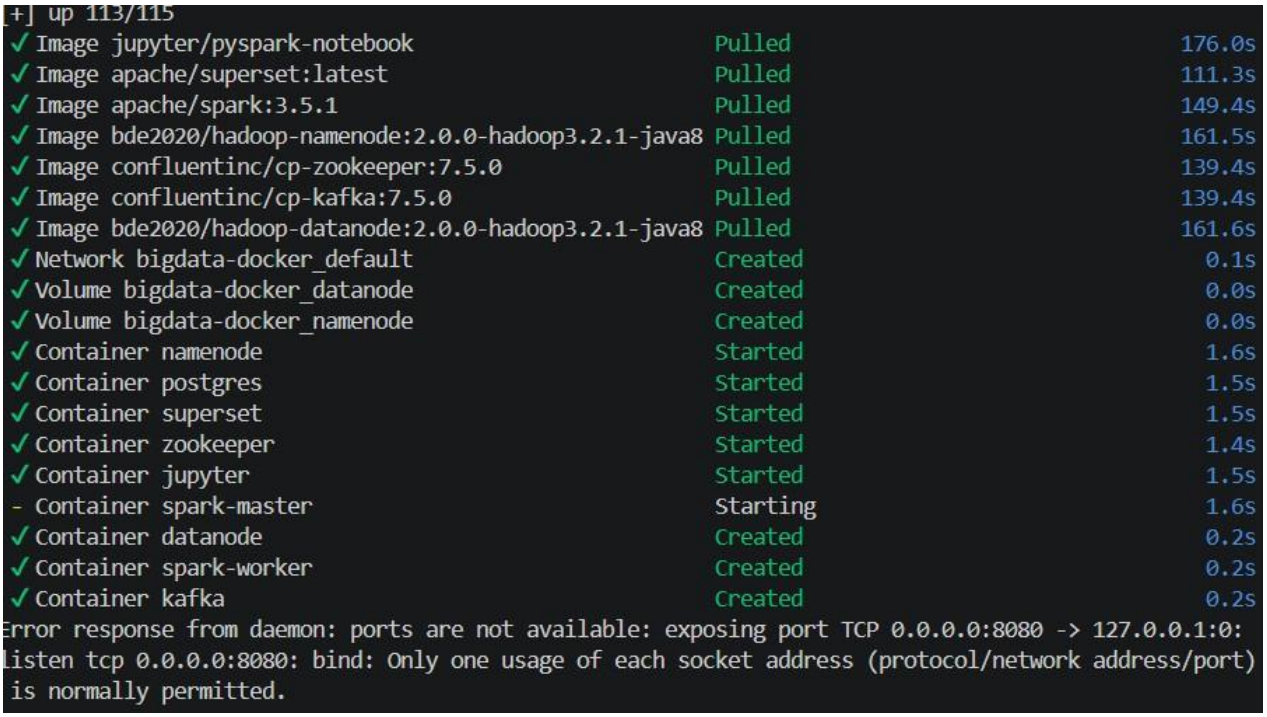
Open PowerShell:

Run:

Invoke-WebRequest `
-Uri "https://jdbc.postgresql.org/download/postgresql-42.7.3.jar" ` -
OutFile "jars/postgresql.jar"

TASK 5 — Start Docker Services

Run: docker compose up -d



TASK 6 — Verify Running Containers

Run: docker ps

You should see:

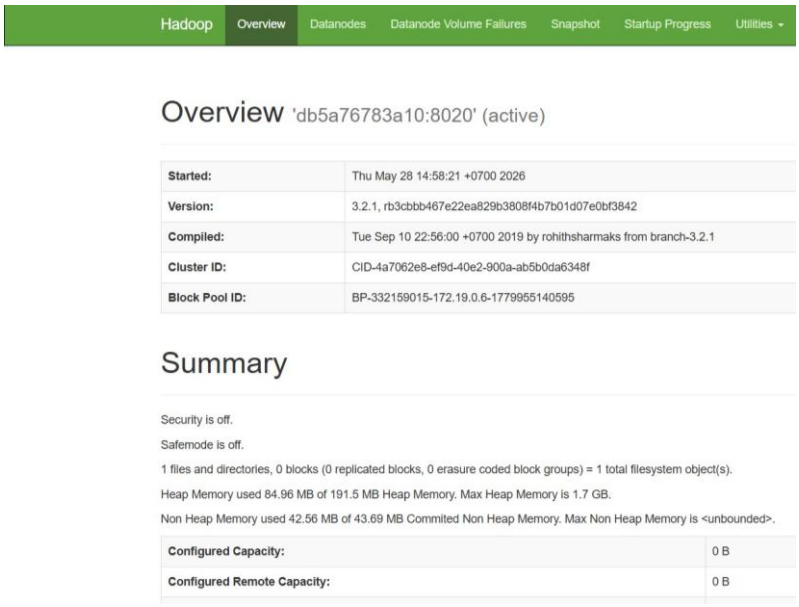
- namenode
- datanode
- spark-master
- spark-worker
- postgres
- jupyter
- superset
- kafka
- zookeeper

```
                                        NAMES
25d8bc84d2f7   postgres:14                                                  "docker-entrypoint.s…"   About
 a minute ago   Up 29 seconds                  0.0.0.0:5433->5432/tcp, [::]:5433->5432/tcp
                                        postgres
a9dea9ba6751   jupyter/pyspark-notebook                                     "tini -g -- start-no…"   About
 a minute ago   Up 29 seconds (healthy)        0.0.0.0:8888->8888/tcp, [::]:8888->8888/tcp
                                        jupyter
82bd87350365   apache/superset:latest                                       "/bin/sh -c ' pip in…"   About
 a minute ago   Up 29 seconds (health: starting)   0.0.0.0:8088->8088/tcp, [::]:8088->8088/tcp
                                        superset
c56fd957846e   confluentinc/cp-zookeeper:7.5.0                              "/etc/confluent/dock…"   About
 a minute ago   Up 29 seconds                  0.0.0.0:2181->2181/tcp, [::]:2181->2181/tcp
                                        zookeeper
db5a76783a10   bde2020/hadoop-namenode:2.0.0-hadoop3.2.1-java8             "/entrypoint.sh /run…"   About
 a minute ago   Up 29 seconds (health: starting)   0.0.0.0:9000->9000/tcp, [::]:9000->9000/tcp, 0.0.
0.0:9870->9870/tcp, [::]:9870->9870/tcp       namenode
57a6f10644fc   nginx:1.25-alpine                                            "/docker-entrypoint.…"   30 ho
urs ago        Up 3 minutes (healthy)         0.0.0.0:80->80/tcp, [::]:80->80/tcp
                                        astron-agent-nginx
94e2f6efaa05   ghcr.io/iflytek/astron-agent/core-workflow:latest            "uv run workflow/mai…"   30 ho
```

TASK 7 — Open Big Data Services

Service	URL
Hadoop HDFS	http://localhost:9870
Spark UI	http://localhost:8080
Jupyter Notebook	http://localhost:8888
Superset	http://localhost:8088

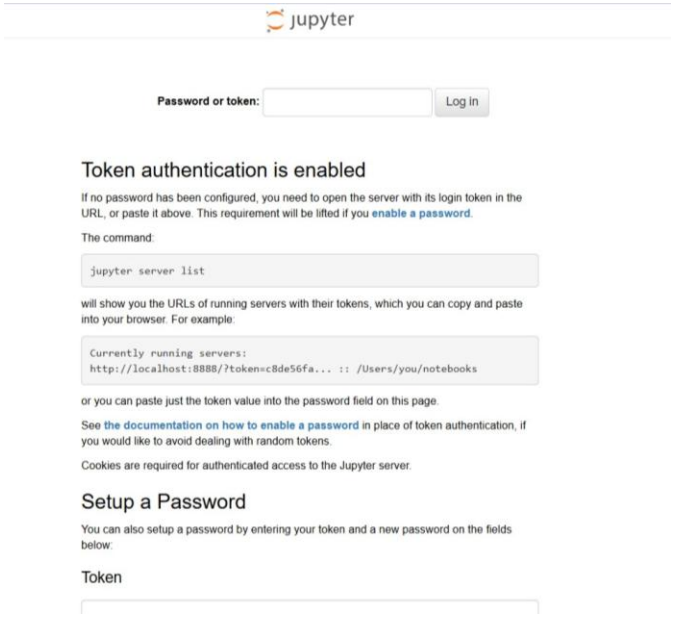
- **Hadoop HDFS**



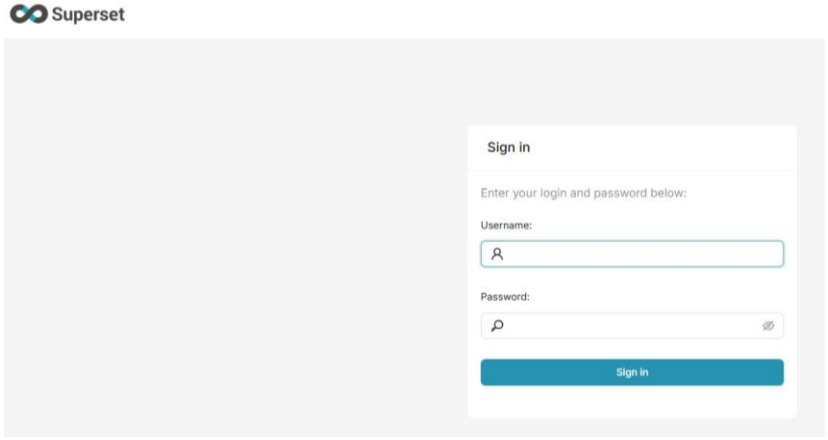
- **Spark UI**



- **Jupyter Notebook**



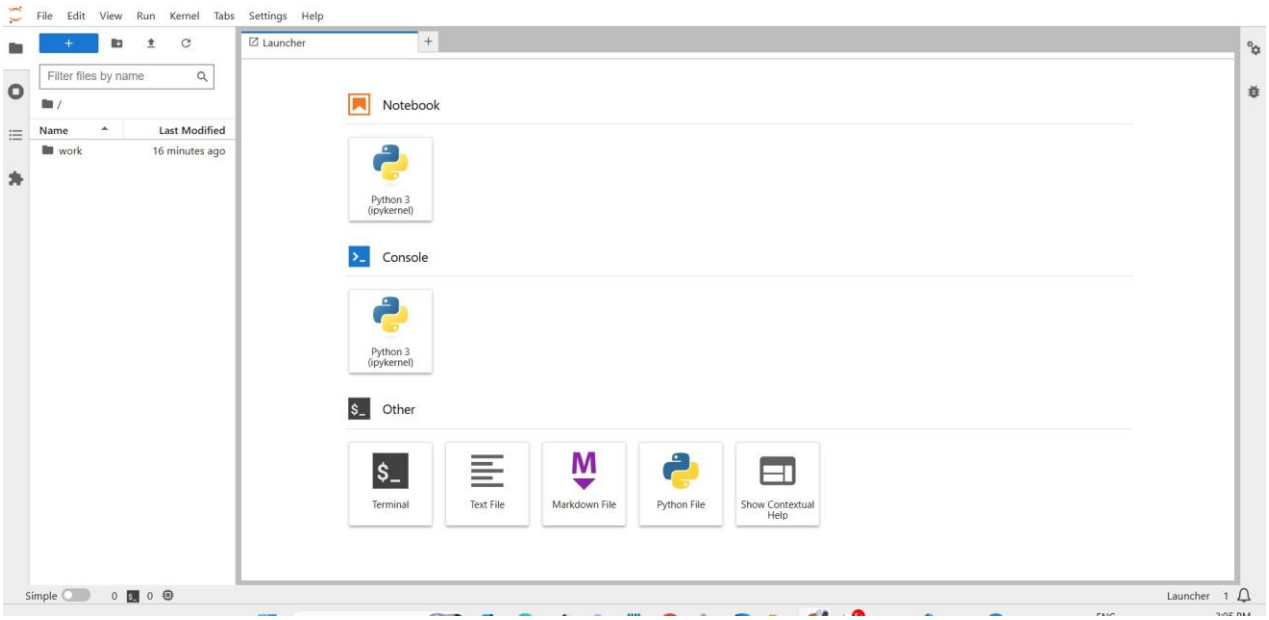
- **Superset**



TASK 8 — Login Credentials

Jupyter Notebook

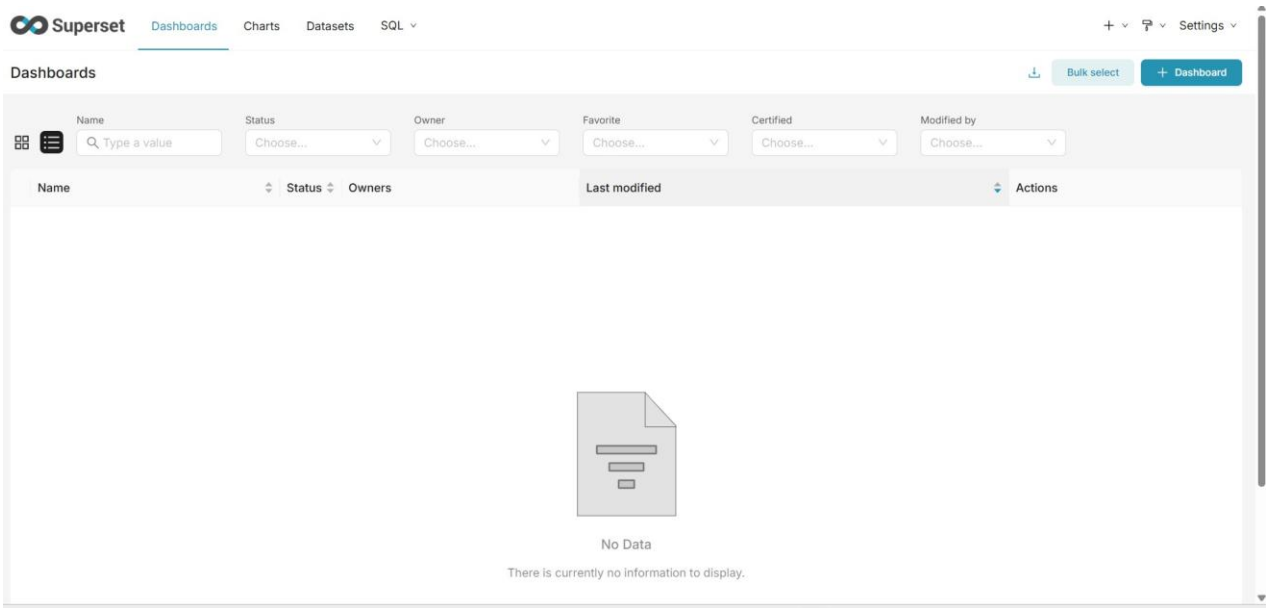
Password/Token: mypassword



Superset

Username: admin

Password: admin123

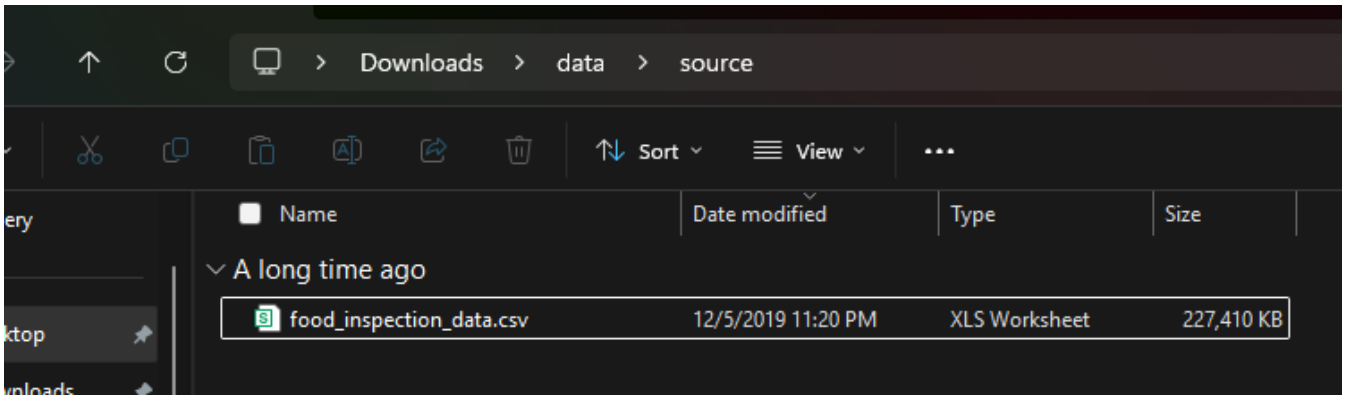


TASK 9 — Download Dataset

Open: https://www.kaggle.com/datasets/chicago/chicago-food-inspections
Download: food-inspections.csv

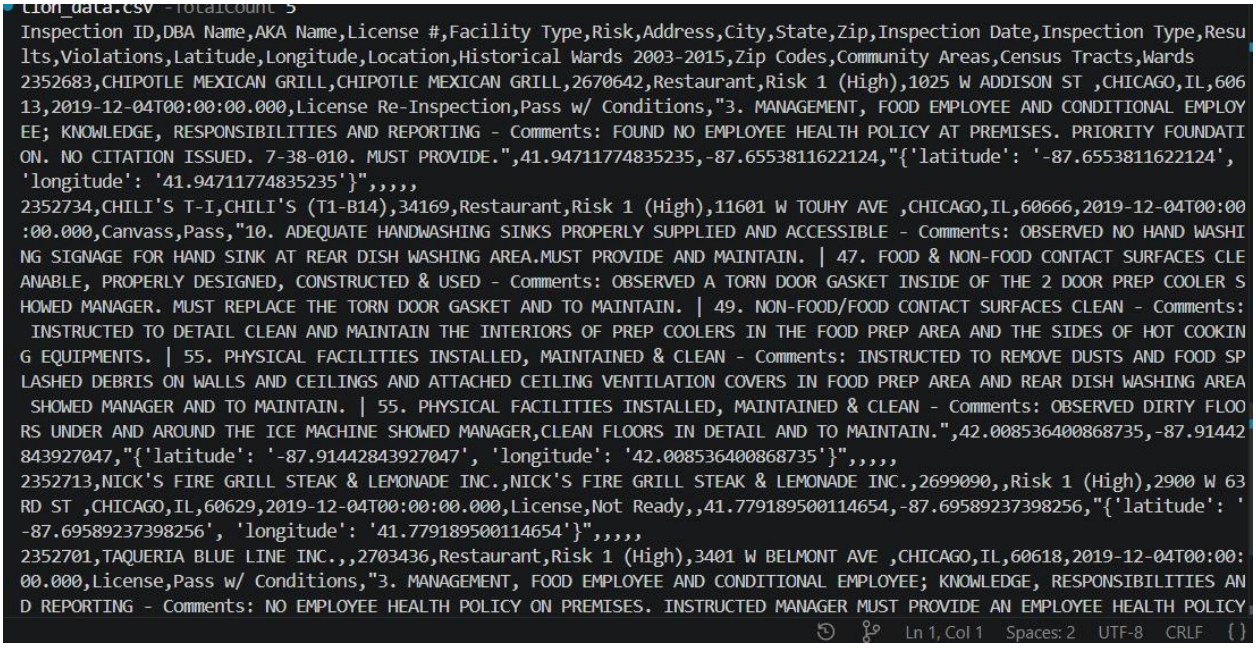
Rename to: **food_inspection_data.csv**

Move file into: **data/source/**



• Verify Dataset

Run: Get-Content data/source/food_inspection_data.csv -TotalCount 5
You should see CSV columns.



TASK 10 — Create Spark Analytics Script

Create file: spark/jobs/process.py
Paste the following code:

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col

spark = SparkSession.builder \
    .appName("FoodInspectionAnalytics") \
```

```
    .getOrCreate()

print("Reading CSV file...")

# Read CSV  df =
spark.read.csv(
    "/opt/spark-data/source/food_inspection_data.csv",
    header=True,  inferSchema=True
)

print("CSV loaded successfully")

# Data Cleaning
clean_df = df.dropna()

clean_df = clean_df.filter(
    col("DBA Name").isNotNull()
)

clean_df.createOrReplaceTempView("violations")

# Analytics Queries  print("Top restaurants
with most inspections")

top_restaurants = spark.sql("""
SELECT
    `DBA Name` AS restaurant_name,
COUNT(*) AS total_inspections
FROM violations
GROUP BY `DBA Name`
ORDER BY total_inspections DESC
LIMIT 10
""")  top_restaurants.show()

print("Inspection risk

distribution")  risk_distribution =

spark.sql("""

SELECT
    Risk,
    COUNT(*) AS total
FROM violations
GROUP BY Risk
```

```
ORDER BY total DESC
""")

risk_distribution.show()
print("Top failure results")
failure_results = spark.sql("""
SELECT
    Results,
    COUNT(*) AS total
FROM violations
GROUP BY Results
ORDER BY total DESC
""")
failure_results.show()

# Save CSV Output

top_restaurants.coalesce(1).write \
    .mode("overwrite") \
    .option("header", "true") \
    .csv("/opt/spark-data/target/output")

# Save Parquet Output

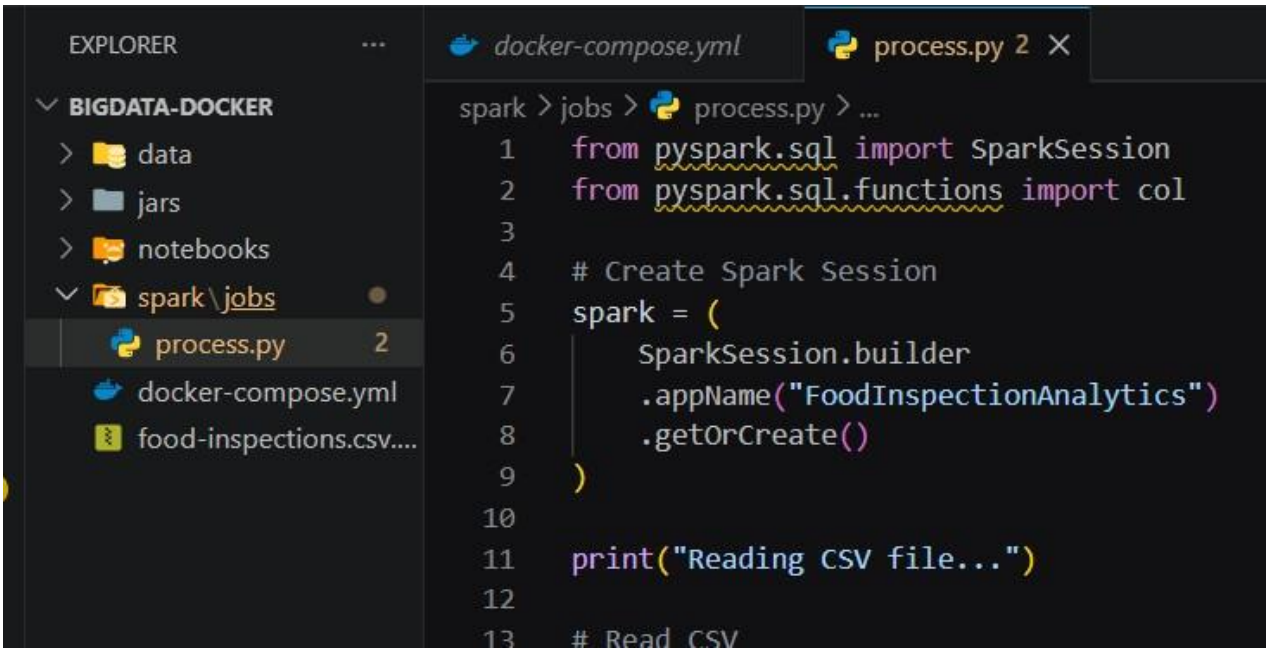
clean_df.write.mode("overwrite").parquet( "/opt/spark-
    data/target/parquet"
)

# Save to PostgreSQL  print("Writing

results to PostgreSQL...")

top_restaurants.write \
    .format("jdbc") \
    .option("url", "jdbc:postgresql://postgres:5432/metastore") \
    .option("dbtable", "top_restaurants") \
    .option("user", "hive") \
    .option("password", "hive") \
    .option("driver", "org.postgresql.Driver") \
    .mode("overwrite") \
    .save()  print("PostgreSQL write

complete")  spark.stop()
```



● Run Spark Analytics Job

Run: **docker exec -it spark-master /opt/spark/bin/spark-submit /opt/spark/jobs/process.py**

```
26/05/28 08:17:42 INFO DAGScheduler: waiting: Set()
26/05/28 08:17:42 INFO DAGScheduler: failed: Set()
26/05/28 08:17:42 INFO SparkContext: Starting job: save at <unknown>:0
26/05/28 08:17:42 INFO DAGScheduler: Job 9 finished: save at <unknown>:0, took 0.000377 s
PostgreSQL write complete
26/05/28 08:17:42 INFO SparkContext: SparkContext is stopping with exitCode 0.
26/05/28 08:17:42 INFO SparkUI: Stopped Spark web UI at http://9462f52c75f9:4040
26/05/28 08:17:42 INFO MapOutputTrackerMasterEndpoint: MapOutputTrackerMasterEndpoint stopped!
26/05/28 08:17:42 INFO MemoryStore: MemoryStore cleared
26/05/28 08:17:42 INFO BlockManager: BlockManager stopped
26/05/28 08:17:42 INFO BlockManagerMaster: BlockManagerMaster stopped
26/05/28 08:17:42 INFO OutputCommitCoordinator$OutputCommitCoordinatorEndpoint: OutputCommitCoordinator stopped!
26/05/28 08:17:42 INFO SparkContext: Successfully stopped SparkContext
26/05/28 08:17:42 INFO ShutdownHookManager: Shutdown hook called
26/05/28 08:17:42 INFO ShutdownHookManager: Deleting directory /tmp/spark-6c559ed1-baf1-488a-a6e8-326659640165
26/05/28 08:17:42 INFO ShutdownHookManager: Deleting directory /tmp/spark-20690a1b-afb2-4f70-b2c6-4a0aa552f3dd
26/05/28 08:17:42 INFO ShutdownHookManager: Deleting directory /tmp/spark-20690a1b-afb2-4f70-b2c6-4a0aa552f3dd/pyspark-d6590498-828
4c94-9e27-1935dc239126

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug spark-master
    Learn more at https://docs.docker.com/go/debug-cli/
```

● Verify Output Files

Run: **dir data\target\output**

You should see: **_SUCCESS part-00000...**

```
Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----         5/28/2026   3:18 PM             12 .part-00000-2e1754e0-15f7-4d9d-9675-fa3468558271-c000.csv.crc
-a----         5/28/2026   3:18 PM              8 ._SUCCESS.crc
-a----         5/28/2026   3:18 PM             34 part-00000-2e1754e0-15f7-4d9d-9675-fa3468558271-c000.csv
-a----         5/28/2026   3:18 PM              0 _SUCCESS
```

● View Analytics Results

Run: **Get-Content data/target/output/part-*.csv**

```
restaurant_name,total_inspections
SUBWAY,2699
DUNKIN DONUTS,1401
MCDONALD'S,575
7-ELEVEN,435
MCDONALDS,333
CHIPOTLE MEXICAN GRILL,312
POTBELLY SANDWICH WORKS LLC,275
CORNER BAKERY CAFE,242
POTBELLY SANDWICH WORKS,238
DUNKIN DONUTS/BASKIN ROBBINS,233
```

TASK 11 — Verify PostgreSQL Data

Run: **docker exec -it postgres psql -U hive -d metastore**

```
 metastore
psql (14.23 (Debian 14.23-1.pgdg13+1))
Type "help" for help.

metastore=#
```

Inside PostgreSQL: SELECT * FROM top_restaurants;

```
metastore=# SELECT * FROM top_restaurants;
       restaurant_name        | total_inspections
------------------------------+-------------------
 SUBWAY                       |              2699
 DUNKIN DONUTS                |              1401
 MCDONALD'S                   |               575
 7-ELEVEN                     |               435
 MCDONALDS                    |               333
 CHIPOTLE MEXICAN GRILL       |               312
 POTBELLY SANDWICH WORKS LLC  |               275
 CORNER BAKERY CAFE           |               242
 POTBELLY SANDWICH WORKS      |               238
 DUNKIN DONUTS/BASKIN ROBBINS |               233
(10 rows)
```

For exit enter the command : **\q**

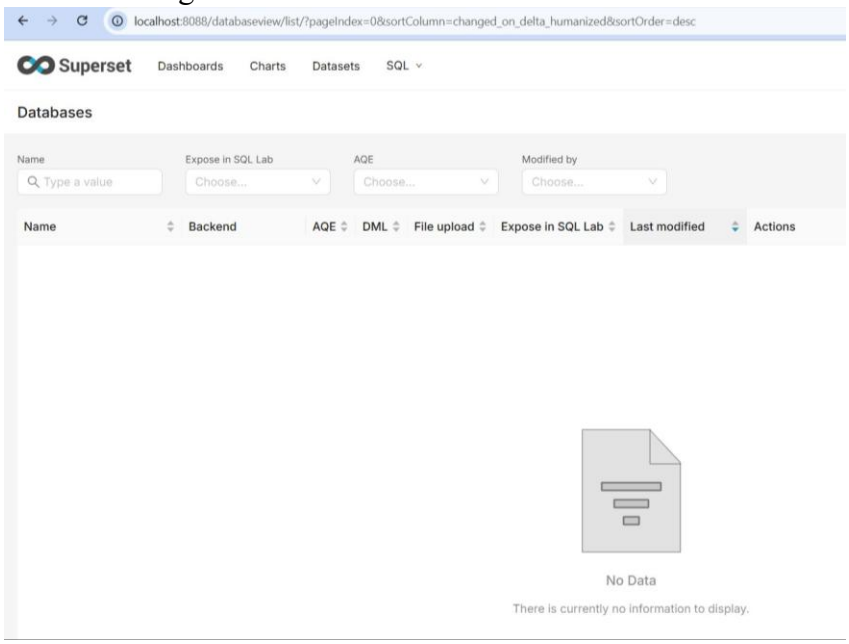
```
metastore=# \q

What's next:
    Try Docker Debug for seamless, persistent debugging to
    Learn more at https://docs.docker.com/go/debug-cli/
```

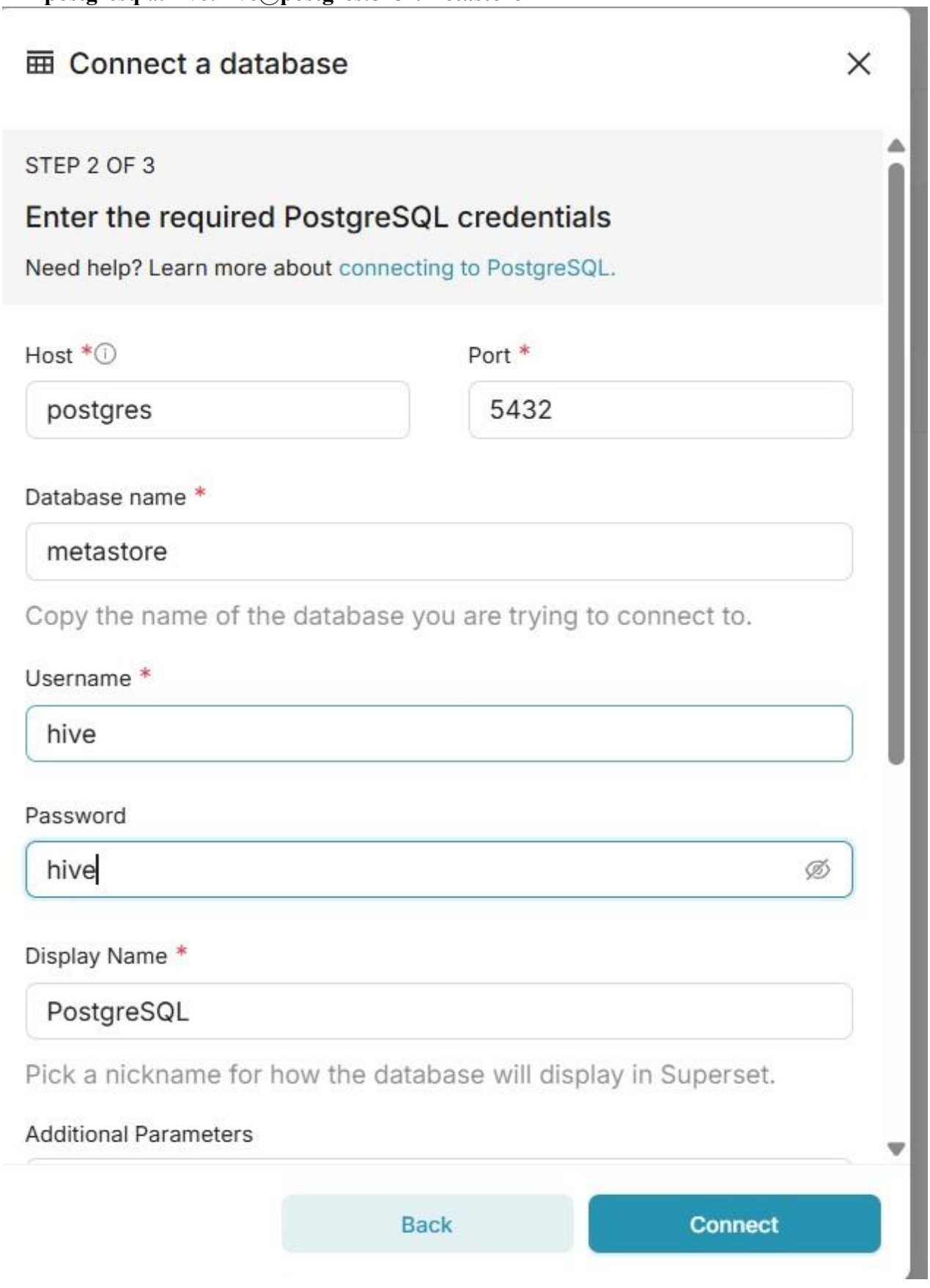
TASK 12- Connect PostgreSQL to Superset

Open: **http://localhost:8088**

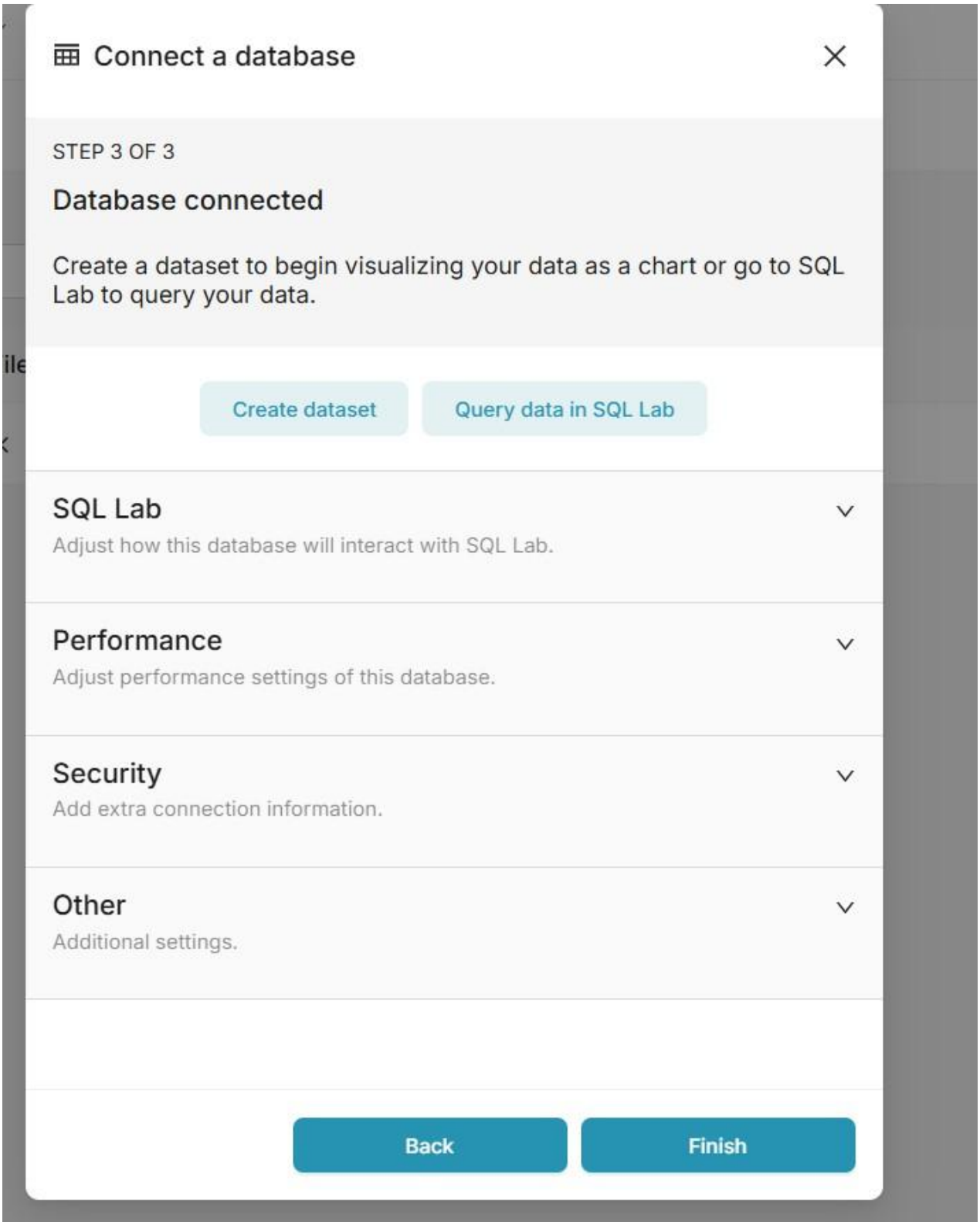
Go to: Settings → Database Connections → + Database



Use: **postgresql://hive:hive@postgres:5432/metastore**



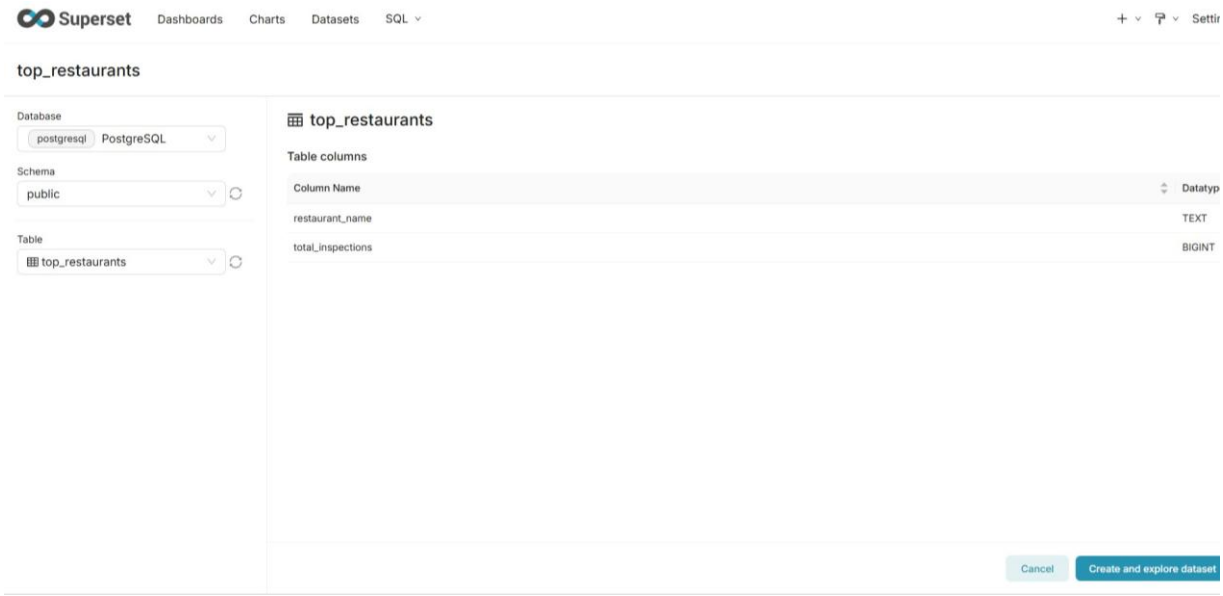
Test Connection and Save.

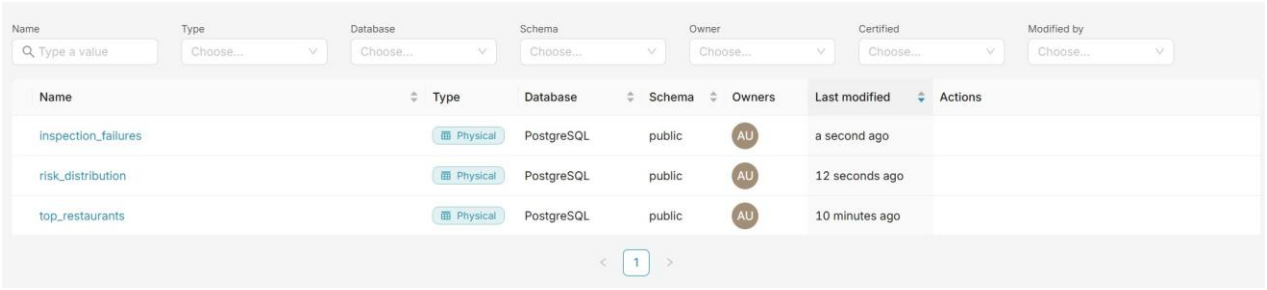


TASK 13 — Create Superset Dashboard

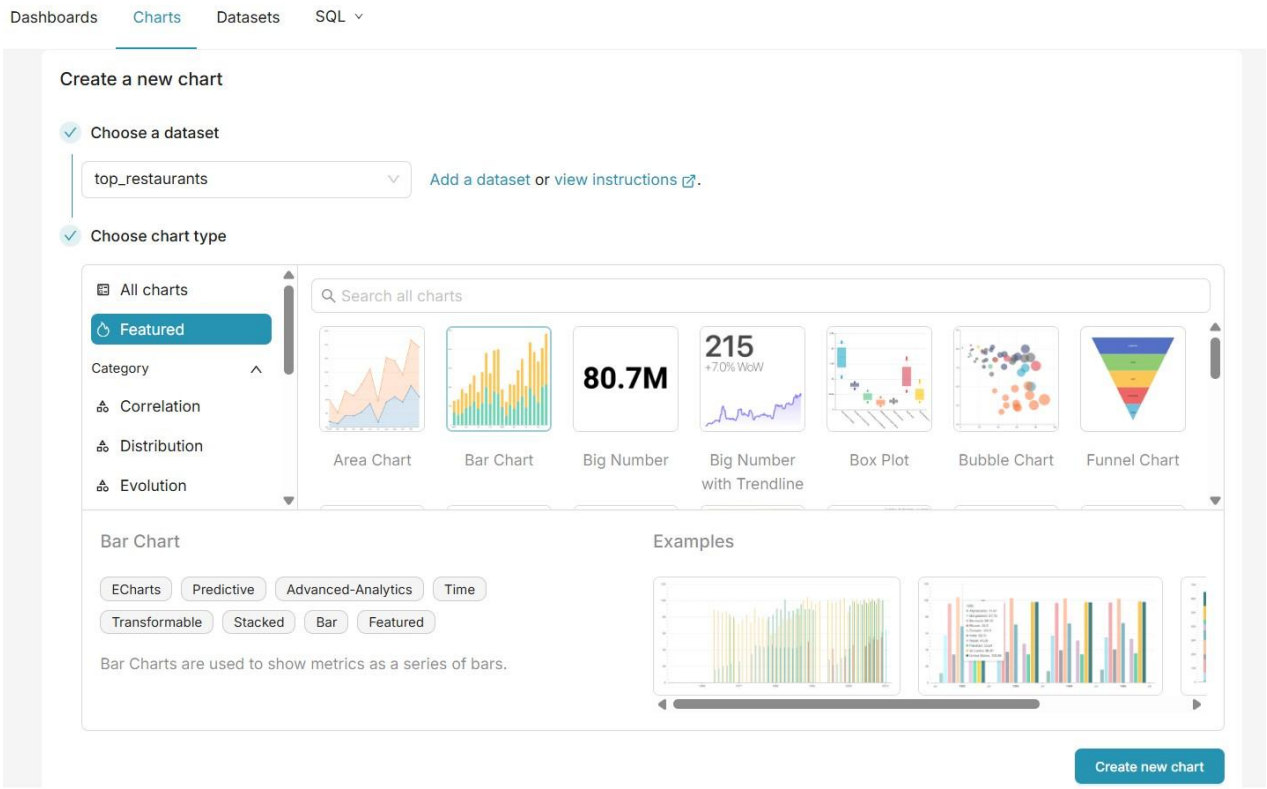
- Create:

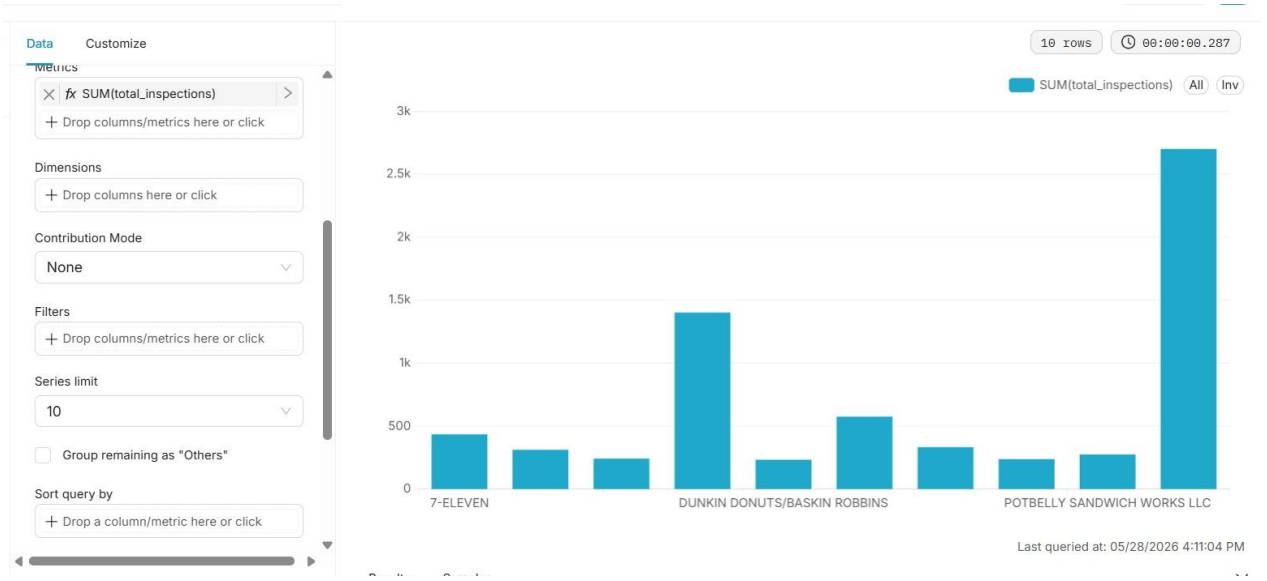
Dataset,

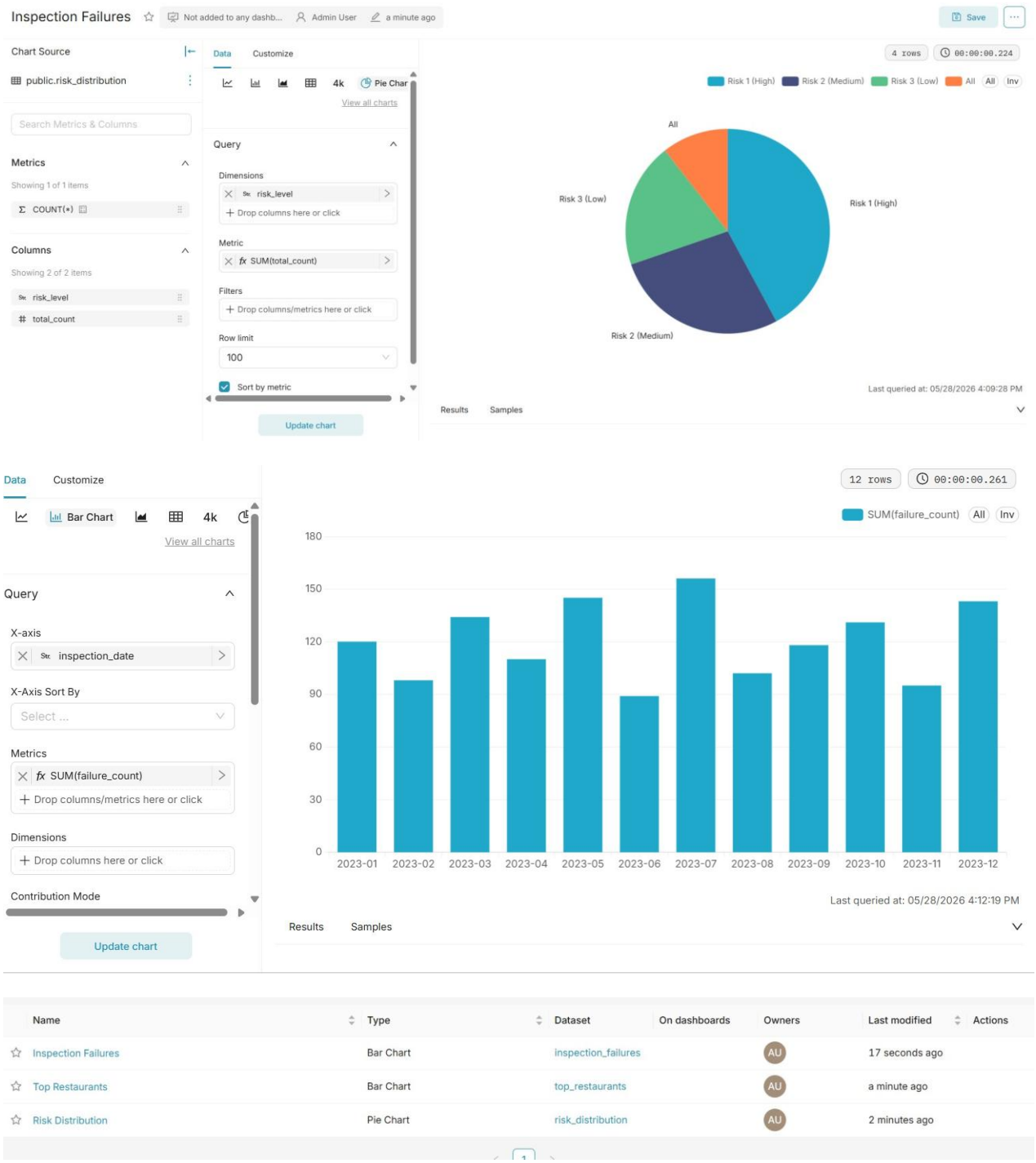




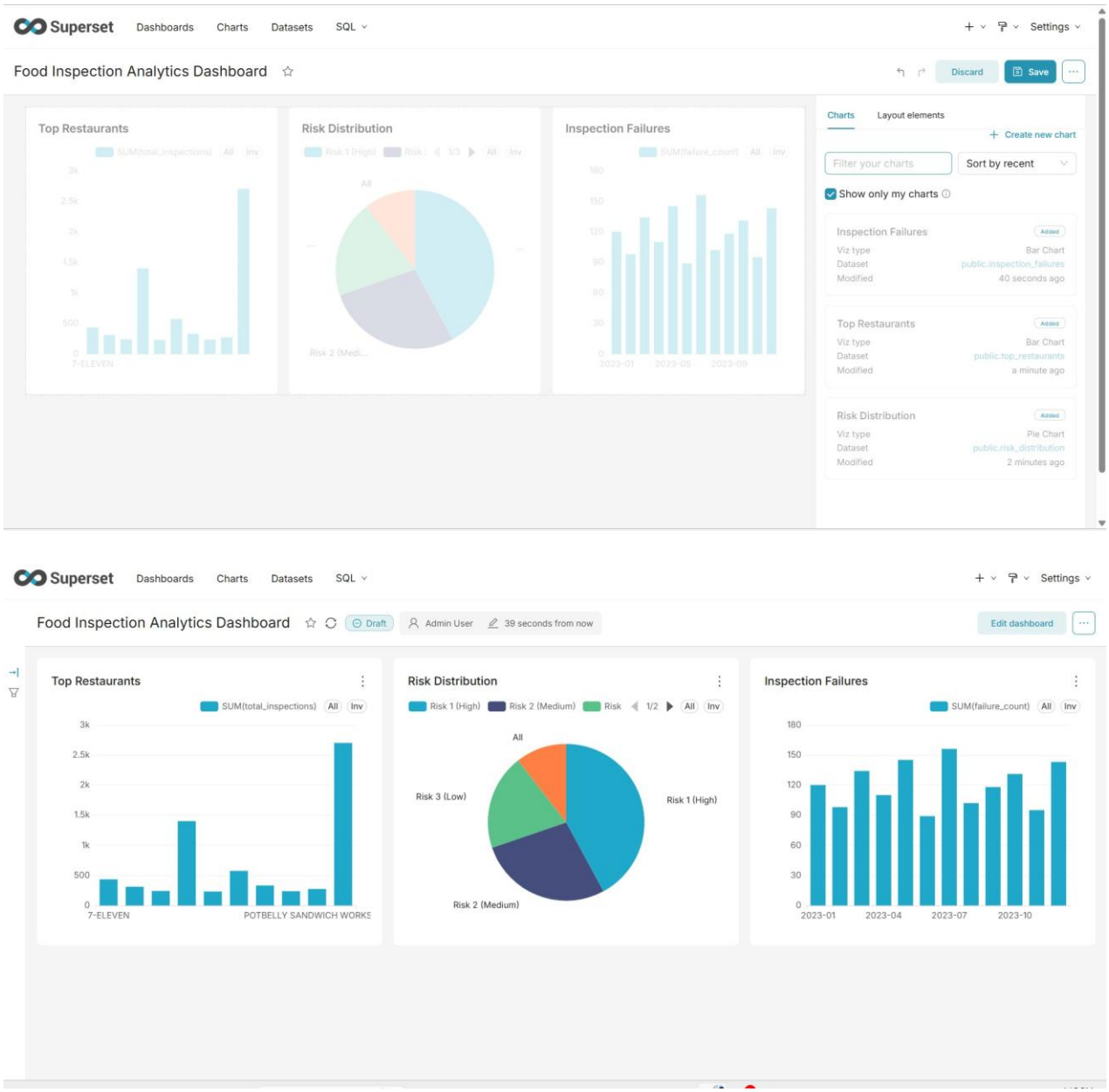
Charts ,


Dashboards
Charts
Datasets
SQL
Create a new chart
Choose a dataset
top_restaurants
Add a dataset or view instructions.
Choose chart type
All charts
Featured
Category
Correlation
Distribution
Evolution
Search all charts
80.7M
215
+7.0% WoW
Area Chart
Bar Chart
Big Number
Big Number with Trendline
Box Plot
Bubble Chart
Funnel Chart
Bar Chart
ECharts
Predictive
Advanced-Analytics
Time
Transformable
Stacked
Bar
Featured
Bar Charts are used to show metrics as a series of bars.
Examples
Create new chart


Data
Customize
Metrics
SUM(total_inspections)
Drop columns/metrics here or click
Dimensions
Drop columns here or click
Contribution Mode
None
Filters
Drop columns/metrics here or click
Series limit
10
Group remaining as "Others"
Sort query by
Drop a column/metric here or click
10 rows
00:00:00.287
SUM(total_inspections)
All
Inv
3k
2.5k
2k
1.5k
1k
500
0
7-ELEVEN
DUNKIN DONUTS/BASKIN ROBBINS
POTBELLY SANDWICH WORKS LLC
Last queried at: 05/28/2026 4:11:04 PM


Inspection Failures
Not added to any dashb...
Admin User
a minute ago
Save
Chart Source
public.risk_distribution
Search Metrics & Columns
Metrics
Showing 1 of 1 items
COUNT(*)
Columns
Showing 2 of 2 items
risk_level
total_count
Data
Customize
4k
Pie Char
View all charts
Query
Dimensions
risk_level
Drop columns here or click
Metric
SUM(total_count)
Filters
Drop columns/metrics here or click
Row limit
100
Sort by metric
Update chart
4 rows
00:00:00.224
Risk 1 (High)
Risk 2 (Medium)
Risk 3 (Low)
All
Inv
All
Risk 3 (Low)
Risk 1 (High)
Risk 2 (Medium)
Last queried at: 05/28/2026 4:09:28 PM
Results
Samples
Data
Customize
Bar Chart
4k
View all charts
Query
X-axis
inspection_date
X-Axis Sort By
Select ...
Metrics
SUM(failure_count)
Drop columns/metrics here or click
Dimensions
Drop columns here or click
Contribution Mode
Update chart
12 rows
00:00:00.261
SUM(failure_count)
All
Inv
180
150
120
90
60
30
0
2023-01
2023-02
2023-03
2023-04
2023-05
2023-06
2023-07
2023-08
2023-09
2023-10
2023-11
2023-12
Last queried at: 05/28/2026 4:12:19 PM
Results
Samples
Name
Type
Dataset
On dashboards
Owners
Last modified
Actions
Inspection Failures
Bar Chart
inspection_failures
AU
17 seconds ago
Top Restaurants
Bar Chart
top_restaurants
AU
a minute ago
Risk Distribution
Pie Chart
risk_distribution
AU
2 minutes ago

Dashboard



- Suggested dashboard name: **Food Inspection Analytics Dashboard**

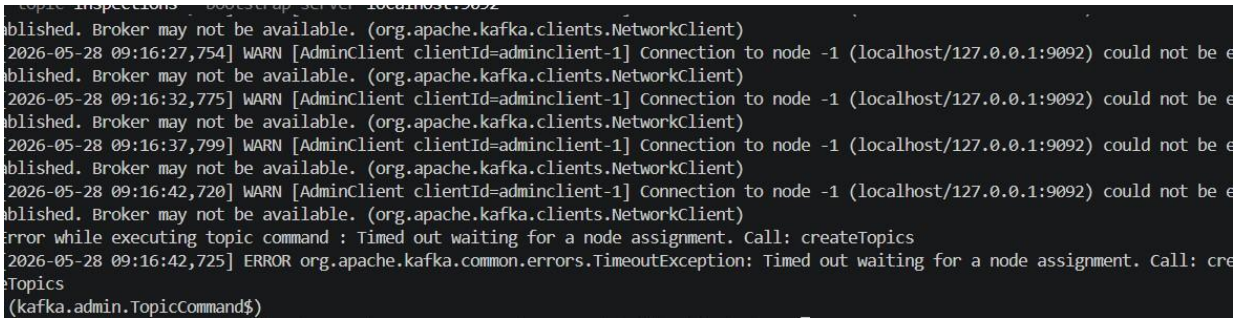
Suggested charts:

- Top Restaurants
- Risk Distribution
- Inspection Failures

TASK 14 — Kafka Streaming Analytics

- ## Create Kafka Topic

Run: **docker exec kafka kafka-topics --create --topic inspections --bootstrap-server localhost:9092**



● Start Kafka Producer

Run: **docker exec -it kafka kafka-console-producer --topic inspections --bootstrap-server localhost:9092**

Send sample events:
Restaurant A,Risk 1,Fail
Restaurant B,Risk 2,Pass



● Start Kafka Consumer

Open another terminal:

Run: **docker exec -it kafka kafka-console-consumer --topic inspections --from-beginning --bootstrap-server localhost:9092**

You should see live streaming events.



TASK 15 — Create Spark Streaming Job

Create file: spark/jobs/streaming.py

Paste the following code

```
from pyspark.sql import SparkSession

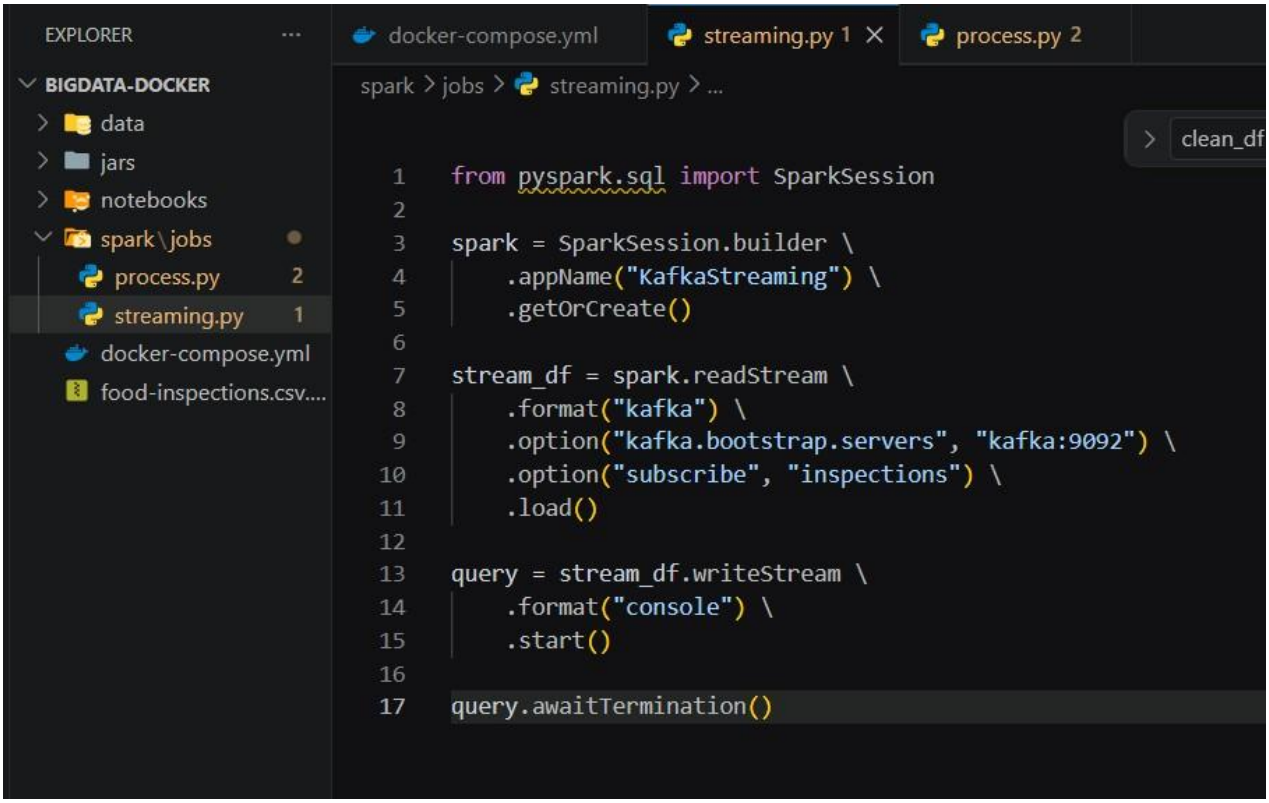
spark = SparkSession.builder \
    .appName("KafkaStreaming") \
```

```
    .getOrCreate()

stream_df = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "kafka:9092") \
    .option("subscribe", "inspections") \
    .load()

query = stream_df.writeStream \
    .format("console") \
    .start()

query.awaitTermination()
```



TASK 16 — Run Spark Streaming Job

Run: **docker exec -it spark-master /opt/spark/bin/spark-submit /opt/spark/jobs/streaming.py** You should observe real-time Kafka data processing.

```
k-submit `
>>   --packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.1 `
>>   /opt/spark/jobs/streaming.py
    },
    "numInputRows" : 0,
    "inputRowsPerSecond" : 0.0,
    "processedRowsPerSecond" : 0.0,
    "metrics" : {
      "avgOffsetsBehindLatest" : "0.0",
      "maxOffsetsBehindLatest" : "0",
      "minOffsetsBehindLatest" : "0"
    }
  } ],
  "sink" : {
    "description" : "org.apache.spark.sql.execution.streaming.ConsoleTable$@537999c4",
    "numOutputRows" : 0
  }
}
26/05/28 09:31:49 INFO MicroBatchExecution: Streaming query has been idle and waiting
```

TASK 17 — Jupyter Notebook Analytics

Open: http://localhost:8888

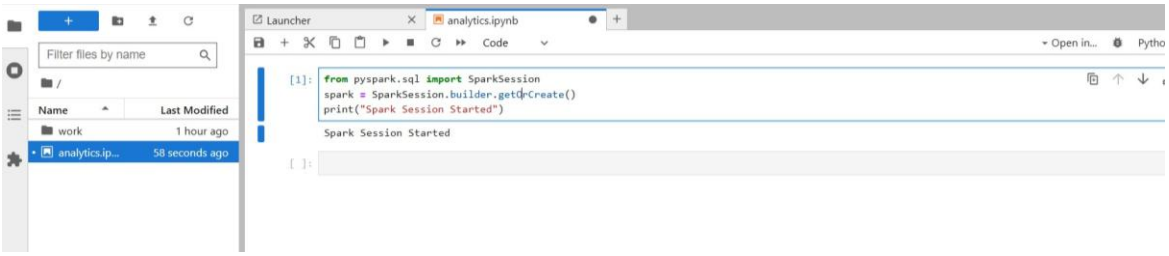
Login using: mypassword

Create notebook: analytics.ipynb

Paste the following code.

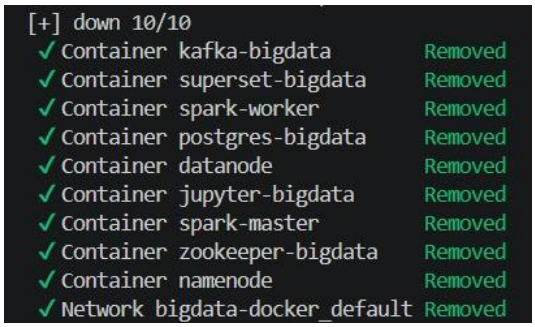
```
from pyspark.sql import SparkSession
spark = SparkSession.builder.getOrCreate()
print("Spark Session Started")
```

Run notebook cell.



- ## Stop Docker Services

Run: docker compose down



Remove Docker Volumes

Run: **docker compose down -v**

